

Python Deep Dive

Architecture, Syntax, Memory & I/O

Mechanics

BEGINNER TO ADVANCED INTERNALS

The DNA of Python

Core Identity & Philosophy

Python vs. The Modern World

Interpreted Core

Python runs via the **Python Virtual Machine (PVM)**. Code is compiled to **Bytecode** (.pyc), then interpreted line-by-line. This ensures platform independence but slower execution than pure C++.

Multi-Paradigm

Supports **Object-Oriented** (Classes), **Procedural** (Functions), and **Functional** (Lambda, Map) programming. You pick the tool that fits the problem.

Dynamic Typing: Types are resolved at **Runtime**. Variables are labels, not memory buckets.

The Basics

Syntax and Standard Operations

Writing Your First Lines



The print() Function

Python's way of talking to you. It executes code line-by-line, top to bottom.

```
print("Hello World!")
```



Comments

Notes for humans that Python ignores. Essential for documenting logic.

```
# This is a comment  
# Python ignores this
```



Quick Task

Try printing your name, favorite food, and a fun fact on 3 separate lines.

Input & Output

Interactive Python uses `input()` to grab data. **Warning:**

Everything it catches is a String!

```
age = int(input("Age: "))
```

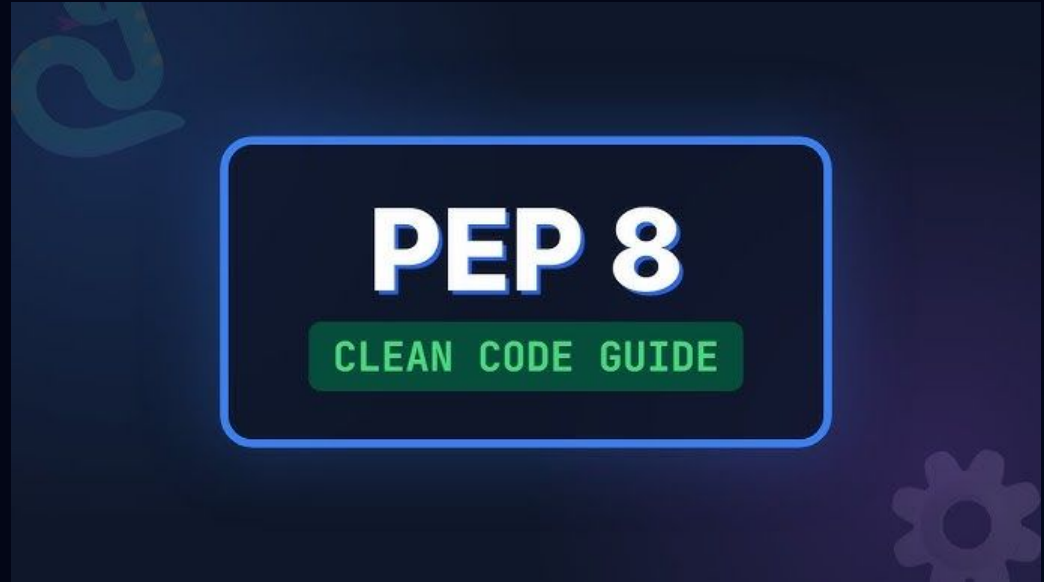
Use **f-strings** for clean output:

```
print(f"You are {age}!")
```

Syntax Guardrails & PEP 8

Do's and Don'ts

- ✓ Use **4 spaces** for indentation (No tabs).
- ✓ snake_case for variables/functions.
- ✓ PascalCase for Classes only.
- ✗ Avoid I, O, l as single names.
- ✗ Never use semicolons ; for ending.



Variables & Data Types

Encyclopedia of Objects

Variables: Memory "Name Tags"

They aren't just boxes...

In most languages, variables are boxes. In Python, variables are **pointers** or "Name Tags" attached to objects in memory.

```
x = 10
y = x # y points to the same 10!
```

When you change `x = 20`, you just move the 'x' tag to a new number in memory.

 Memory pointers visualization

Variable Lifecycle & RAM

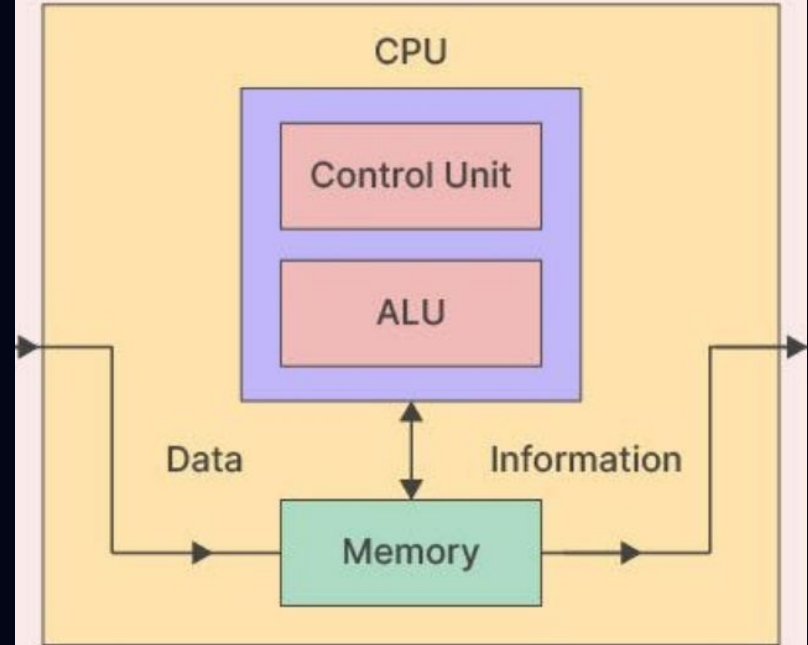
Python uses **Reference Counting** and **Garbage Collection**. Variables are merely pointers to objects.

`id(x)`: Returns the C-level memory address.

`type(x)`: Returns the class definition.

When reference count hits zero, Python reclaims the memory automatically.

Basic Architecture of a Computer



The id() Function

id()

Memory Location

Checking the Address

Every object in Python gets a unique ID. You can see it with id().

```
a = 100  
print(id(a)) # Outputs memory address
```

If two variables have the same ID, they are literally pointing to the same spot in your RAM.

Mutability

Mutable vs. Immutable Objects

The Golden Rule of Data

Immutable

Locked. Cannot be changed after creation. If you "modify" it, Python creates a new one.

- int, float, str
- bool, tuple

Mutable

Flexible. Can be changed in-place without moving the name tag to a new address.

- list, dict
- set

Complete Data Type Taxonomy

Type	Mutability	Memory / Logic Detail	Primary Use Case
int / float	Immutable	Address changes on update (New Object).	Calculations, indexing.
str	Immutable	Internal string interning for optimization.	Text processing.
bool	Immutable	Constants; only two exist in memory.	Conditional logic.
list	Mutable	Modified in-place; same address.	Ordered collections.
dict	Mutable	Hashed keys; extremely fast retrieval.	Key-Value pairs.
tuple	Immutable	Faster and more memory efficient than lists.	Fixed collections.
set	Mutable	Unordered; unique items only.	Math set ops (Union).

The Comprehensive Operator Toolbox

Category	Operators	Example & Precedence
Arithmetic	+ - * / // % **	10 // 3 is 3 (Floor). 2 ** 3 is 8.
Comparison	== != > < >= <=	x == y returns Boolean result.
Logical	and or not	(a > b) and (c < d) (Binary).
Identity	is, is not	Checks memory address (id). x is y
Membership	in, not in	Checks sequence. 'a' in "apple"
Assignment	= += -= *= /=	Lowest precedence. x += 5

Basic Arithmetic & String Ops

Math Details

Floor Div ($10//3 = 3$) vs Modulo ($10\%3 = 1$). Exponent ($2**3 = 8$).

String Power

`"Go" * 3 = "GoGoGo"`

Concatenation: `"A" + "B"`

Repetition: `"A" * 5`

Slicing: `s[1:4]` gets chars at index 1, 2, 3.

I/O Mechanics: From OS to Screen

OS Level (Hardware)

Managed via **File Descriptors**:

- FD 0: STDIN (Keyboard)
- FD 1: STDOUT (Screen)
- FD 2: STDERR (Errors)

Input travels through an **OS Buffer** before the app reads it.

Program Level

`input()` reads from STDIN and returns a str.

`print()` writes to STDOUT.

Buffers are "flushed" to screen automatically or via `flush=True`.

Q&A

Deep dive complete. Any questions on the
internals?

```
>>> sys.exit(0)  
# Success: Deep Dive Completed
```

Image Sources



https://miro.medium.com/0*MGwhgis8-7h-MOM0

Source: medium.com



https://miro.medium.com/v2/resize:fit:1000/1*FdJHrZr2xg-k25x_QTdOqA.jpeg

Source: medium.com



<https://iitutor.com/wp-content/uploads/2025/07/IB-MAThematics-AA.webp>

Source: iitutor.com



https://i.ytimg.com/vi/JbedCC6u_fU/hq720.jpg?sqp=-oaymwE7CK4FEIIDSFRyq4qpAyOIARUAAAAAGAEIAADIQj0AgKJD8AEB-AH-CYAC0AWKAgwIABABGBEGXyhyMA8=&rs=AO4CLAz6TU4mQGjCF7ItQM5Au0dzri-gA

Source: www.youtube.com